

BÁO CÁO TỔNG HỢP MÃ NGUỒN TOÀN BỘ CÁC BÀI THỰC HÀNH

Sinh viên thực hiện: Lê Viết Đức

MSSV: 23DTHA4 / 0533

Môn học: Thực hành Bảo mật thông tin nâng cao

Đường dẫn file: `lab-01\ex02_01.py`

```
ten = input("Nhập tên của bạn: ")
tuoi = input("Nhập tuổi của bạn: ")
print("Chào mừng,", ten, "! Bạn", tuoi, "tuổi.")
```

Đường dẫn file: `lab-01\ex02_02.py`

```
ban_kinh = float(input("Nhập bán kính của hình tròn: "))
dien_tich = 3.14 * (ban_kinh ** 2)
print("Diện tích của hình tròn là:", dien_tich)
```

Đường dẫn file: `lab-01\ex02_03.py`

```
so = int(input("Nhập một số nguyên: "))
if so % 2 == 0:
    print(so, "là số chẵn.")
else:
    print(so, "là số lẻ.")
```

Đường dẫn file: `lab-01\ex02_04.py`

```
j = []
for i in range(2000, 3201):
    if (i % 7 == 0) and (i % 5 != 0):
        j.append(str(i))
```

```
print(', '.join(j))
```

Đường dẫn file: `lab-01\ex02_05.py`

```
so_gio_lam = float(input("Nhập số giờ làm việc mỗi tuần: "))
luong_gio = float(input("Nhập thù lao trên mỗi giờ làm việc tiêu chuẩn: "))
gio_tieu_chuan = 44

if so_gio_lam <= gio_tieu_chuan:
    tong_luong = so_gio_lam * luong_gio
else:
    gio_vuot_chuan = so_gio_lam - gio_tieu_chuan
    tong_luong = (gio_tieu_chuan * luong_gio) + (gio_vuot_chuan * luong_gio * 1.5)

print(f"Số tiền thực nhận của nhân viên: {tong_luong}")
```

Đường dẫn file: `lab-01\ex02_06.py`

```
input_str = input("Nhập X, Y: ")
dimensions = [int(x) for x in input_str.split(',')]
rowNum = dimensions[0]
colNum = dimensions[1]
multilist = [[0 for col in range(colNum)] for row in range(rowNum)]

for row in range(rowNum):
    for col in range(colNum):
        multilist[row][col] = row * col

print(multilist)
```

Đường dẫn file: `lab-01\ex02_07.py`

```
print("Nhập các dòng văn bản (Nhập 'done' để kết thúc):")
lines = []
while True:
    line = input()
    if line.lower() == 'done':
        break
    lines.append(line)
```

```
print("\nCác dòng chữ in hoa:")
for line in lines:
    print(line.upper())
```

Đường dẫn file: lab-01\ex02_08.py

```
def chia_het_cho_5(so_nhi_phan):
    so_thap_phan = int(so_nhi_phan, 2)
    return so_thap_phan % 5 == 0

chuoi_nhi_phan = input("Nhập chuỗi số nhị phân (phân cách bởi dấu phẩy): ")
cac_so_nhi_phan = chuoi_nhi_phan.split(',')
so_chia_het_cho_5 = [so for so in cac_so_nhi_phan if chia_het_cho_5(so)]

if len(so_chia_het_cho_5) > 0:
    print("Các số nhị phân chia hết cho 5 là:", ','.join(so_chia_het_cho_5))
else:
    print("Không có số nhị phân nào chia hết cho 5.")
```

Đường dẫn file: lab-01\ex02_09.py

```
def kiem_tra_so_nguyen_to(n):
    if n <= 1:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

number = int(input("Nhập vào số cần kiểm tra: "))
if kiem_tra_so_nguyen_to(number):
    print(number, "là số nguyên tố.")
else:
    print(number, "không phải là số nguyên tố.")
```

Đường dẫn file: lab-01\ex02_10.py

```
def dao_nguoc_chuoi(chuoi):
    return chuoi[::-1]
```

```
input_string = input("Nhập chuỗi cần đảo ngược: ")
print("Chuỗi đảo ngược là:", dao_nguoc_chuoi(input_string))
```

Đường dẫn file: lab-01\ex03\ex03_01.py

```
# Nhập danh sách từ người dùng
input_list = input("Nhập danh sách các số, cách nhau bằng dấu phẩy: ")
numbers = [int(x) for x in input_list.split(',')]
sum_even = sum(x for x in numbers if x % 2 == 0)
print("Tổng các số chẵn trong danh sách là:", sum_even)
```

Đường dẫn file: lab-01\ex03\ex03_02.py

```
input_list = input("Nhập danh sách các số, cách nhau bằng dấu phẩy: ")
numbers = [int(x) for x in input_list.split(',')]
numbers.reverse()
print("Danh sách sau khi đảo ngược:", numbers)
```

Đường dẫn file: lab-01\ex03\ex03_03.py

```
input_list = input("Nhập danh sách các số, cách nhau bằng dấu phẩy: ")
numbers = [int(x) for x in input_list.split(',')]
my_tuple = tuple(numbers)
print("List:", numbers)
print("Tuple tạo từ List:", my_tuple)
```

Đường dẫn file: lab-01\ex03\ex03_04.py

```
input_tuple = input("Nhập các phần tử của tuple, cách nhau bằng dấu phẩy: ")
my_tuple = tuple(input_tuple.split(','))
print("Tuple:", my_tuple)
print("Phần tử đầu tiên:", my_tuple[0])
print("Phần tử cuối cùng:", my_tuple[-1])
```

Đường dẫn file: lab-01\ex03\ex03_05.py

```
input_list = input("Nhập danh sách các phần tử, cách nhau bằng dấu phẩy: ")
elements = input_list.split(',')
frequency = {}
for item in elements:
    item = item.strip()
    frequency[item] = frequency.get(item, 0) + 1
print("Số lần xuất hiện của các phần tử:", frequency)
```

Đường dẫn file: lab-01\ex03\ex03_06.py

```
my_dict = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
print("Dictionary ban đầu:", my_dict)
key_to_delete = input("Nhập khóa cần xóa: ")
if key_to_delete in my_dict:
    del my_dict[key_to_delete]
    print("Dictionary sau khi xóa phần tử:", my_dict)
else:
    print("Khóa không tồn tại trong dictionary.")
```

Đường dẫn file: lab-01\ex04\main.py

```
from student_manager import StudentManager

def show_student_list(students):
    if not students:
        print("\nDanh sách sinh viên trống.")
        return
    print("\n" + "-" * 85)
    print(f"{'ID':<5} | {'Họ và tên':<25} | {'Giới tính':<10} | {'Chuyên ngành':<20} | {'ĐTB':<5} | {'Học lực'}")
    print("-" * 85)
    for s in students:
        print(f"{s.id:<5} | {s.name:<25} | {s.gender:<10} | {s.major:<20} | {s.gpa:<5.1f} | {s.rank}")
        print("-" * 85)

def main():
    manager = StudentManager()

    # Seed some sample data for demonstration
```

```

manager.add_student("Nguyen Van A", "Nam", "ATTT", 8.5)
manager.add_student("Tran Thi B", "Nu", "CNTT", 6.8)
manager.add_student("Le Van C", "Nam", "KTPM", 4.5)

while True:
    print("\n=== CHƯƠNG TRÌNH QUẢN LÝ SINH VIÊN ===")
    print("1. Thêm sinh viên")
    print("2. Cập nhật thông tin sinh viên bởi ID")
    print("3. Xóa sinh viên bởi ID")
    print("4. Tìm kiếm sinh viên theo tên")
    print("5. Sắp xếp sinh viên theo Điểm trung bình (GPA)")
    print("6. Sắp xếp sinh viên theo Chuyên ngành (Major)")
    print("7. Hiển thị danh sách sinh viên")
    print("8. Thoát")

    choice = input("Nhập lựa chọn của bạn (1-8): ").strip()

    if choice == '1':
        print("\n--- THÊM SINH VIÊN MỚI ---")
        name = input("Nhập tên sinh viên: ")
        gender = input("Nhập giới tính: ")
        major = input("Nhập chuyên ngành: ")
        try:
            gpa = float(input("Nhập điểm trung bình (GPA): "))
            if gpa < 0 or gpa > 10:
                print("Lỗi: Điểm GPA phải nằm trong khoảng từ 0 đến 10.")
                continue
            manager.add_student(name, gender, major, gpa)
            print("Thêm sinh viên thành công!")
        except ValueError:
            print("Lỗi: Điểm GPA phải là một số thực.")

    elif choice == '2':
        print("\n--- CẬP NHẬT THÔNG TIN SINH VIÊN ---")
        try:
            student_id = int(input("Nhập ID sinh viên cần cập nhật: "))
            name = input("Nhập tên mới: ")
            gender = input("Nhập giới tính mới: ")
            major = input("Nhập chuyên ngành mới: ")
            gpa = float(input("Nhập điểm GPA mới: "))
            if gpa < 0 or gpa > 10:
                print("Lỗi: Điểm GPA phải nằm trong khoảng từ 0 đến 10.")
                continue
            if manager.update_student(student_id, name, gender, major, gpa):
                print("Cập nhật thông tin sinh viên thành công!")
            else:
                print(f"Không tìm thấy sinh viên với ID = {student_id}")
        except ValueError:
            print("Lỗi: ID phải là số nguyên và GPA phải là số thực.")

    elif choice == '3':
        print("\n--- XÓA SINH VIÊN ---")
        try:
            student_id = int(input("Nhập ID sinh viên cần xóa: "))

```

```

        if manager.delete_student(student_id):
            print("Xóa sinh viên thành công!")
        else:
            print(f"Không tìm thấy sinh viên với ID = {student_id}")
    except ValueError:
        print("Lỗi: ID phải là số nguyên.")

elif choice == '4':
    print("\n--- TÌM KIẾM SINH VIÊN THEO TÊN ---")
    name = input("Nhập tên cần tìm kiếm: ")
    results = manager.search_by_name(name)
    show_student_list(results)

elif choice == '5':
    print("\n--- SẮP XẾP SINH VIÊN THEO GPA (GIẢM DẦN) ---")
    manager.sort_by_gpa()
    show_student_list(manager.get_all_students())

elif choice == '6':
    print("\n--- SẮP XẾP SINH VIÊN THEO CHUYÊN NGÀNH ---")
    manager.sort_by_major()
    show_student_list(manager.get_all_students())

elif choice == '7':
    print("\n--- DANH SÁCH SINH VIÊN ---")
    show_student_list(manager.get_all_students())

elif choice == '8':
    print("\nTạm biệt!")
    break

else:
    print("Lựa chọn không hợp lệ. Vui lòng nhập lại từ 1 đến 8.")

if __name__ == '__main__':
    main()

```

Đường dẫn file: **lab-01\ex04\student.py**

```

class Student:
    def __init__(self, student_id, name, gender, major, gpa):
        self.id = student_id
        self.name = name
        self.gender = gender
        self.major = major
        self.gpa = gpa
        self.rank = self.calculate_rank()

    def calculate_rank(self):

```

```
if self.gpa >= 8.0:
    return "Giỏi"
elif self.gpa >= 6.5:
    return "Khá"
elif self.gpa >= 5.0:
    return "Trung bình"
else:
    return "Yếu"
```

Đường dẫn file: lab-01\ex04\student_manager.py

```
from student import Student

class StudentManager:
    def __init__(self):
        self.students = []
        self.next_id = 1

    def add_student(self, name, gender, major, gpa):
        student = Student(self.next_id, name, gender, major, gpa)
        self.students.append(student)
        self.next_id += 1
        return student

    def update_student(self, student_id, name, gender, major, gpa):
        for s in self.students:
            if s.id == student_id:
                s.name = name
                s.gender = gender
                s.major = major
                s.gpa = gpa
                s.rank = s.calculate_rank()
                return True
        return False

    def delete_student(self, student_id):
        for i, s in enumerate(self.students):
            if s.id == student_id:
                del self.students[i]
                return True
        return False

    def search_by_name(self, name):
        results = []
        for s in self.students:
            if name.lower() in s.name.lower():
                results.append(s)
        return results
```

```
def sort_by_gpa(self):
    self.students.sort(key=lambda x: x.gpa, reverse=True)

def sort_by_major(self):
    self.students.sort(key=lambda x: x.major.lower())

def get_all_students(self):
    return self.students
```

Đường dẫn file: `lab-02\ex01\api.py`

```
from flask import Flask, request, jsonify
from cipher.caesar import CaesarCipher

app = Flask(__name__)
caesar_cipher = CaesarCipher()

@app.route("/", methods=["GET"])
def index():
    return "Caesar Cipher API is running"

@app.route("/api/caesar/encrypt", methods=["POST"])
def caesar_encrypt():
    data = request.json

    if data is None or "plain_text" not in data or "key" not in data:
        return jsonify({"error": "Missing plain_text or key"}), 400

    plain_text = data["plain_text"]

    try:
        key = int(data["key"])
    except (ValueError, TypeError):
        return jsonify({"error": "Key must be an integer"}), 400

    encrypted_text = caesar_cipher.encrypt_text(plain_text, key)
    return jsonify({"encrypted_message": encrypted_text})

@app.route("/api/caesar/decrypt", methods=["POST"])
def caesar_decrypt():
    data = request.json

    if data is None or "cipher_text" not in data or "key" not in data:
        return jsonify({"error": "Missing cipher_text or key"}), 400
```

```
    cipher_text = data["cipher_text"]

    try:
        key = int(data["key"])
    except (ValueError, TypeError):
        return jsonify({"error": "Key must be an integer"}), 400

    decrypted_text = caesar_cipher.decrypt_text(cipher_text, key)
    return jsonify({"decrypted_message": decrypted_text})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Đường dẫn file: lab-02\ex01\requirements.txt

```
Flask>=2.3.2
```

Đường dẫn file: lab-02\ex01\cipher__init__.py

```
from .alphabet import ALPHABET
from .caesar_cipher import CaesarCipher
```

Đường dẫn file: lab-02\ex01\cipher\caesar\alphabet.py

```
from string import ascii_uppercase

ALPHABET = list(ascii_uppercase)
```

Đường dẫn file: `lab-02\ex01\cipher\caesar\caesar_cipher.py`

```
from cipher.caesar import ALPHABET

class CaesarCipher:
    def __init__(self):
        self.alphabet = ALPHABET

    def encrypt_text(self, text: str, key: int) -> str:
        text = text.upper()
        alphabet_len = len(self.alphabet)
        encrypted_text = []

        for letter in text:
            if letter in self.alphabet:
                letter_index = self.alphabet.index(letter)
                output_index = (letter_index + key) % alphabet_len
                encrypted_text.append(self.alphabet[output_index])
            else:
                encrypted_text.append(letter)

        return "".join(encrypted_text)

    def decrypt_text(self, text: str, key: int) -> str:
        text = text.upper()
        alphabet_len = len(self.alphabet)
        decrypted_text = []

        for letter in text:
            if letter in self.alphabet:
                letter_index = self.alphabet.index(letter)
                output_index = (letter_index - key) % alphabet_len
                decrypted_text.append(self.alphabet[output_index])
            else:
                decrypted_text.append(letter)

        return "".join(decrypted_text)
```

Đường dẫn file: `lab-02\ex02\api.py`

```
from flask import Flask, request, jsonify
from cipher.vigenere import VigenereCipher

app = Flask(__name__)
vigenere_cipher = VigenereCipher()
```

```
@app.route("/", methods=["GET"])
def index():
    return "Vigenere API is running"

@app.route("/api/vigenere/encrypt", methods=["POST"])
def vigenere_encrypt_api():
    data = request.json

    if data is None or "plain_text" not in data or "key" not in data:
        return jsonify({"error": "Missing plain_text or key"}), 400

    plain_text = data["plain_text"]
    key = data["key"]

    if not isinstance(key, str) or key == "":
        return jsonify({"error": "Key must not be empty"}), 400

    if not key.isalpha():
        return jsonify({"error": "Key must contain only letters"}), 400

    encrypted_text = vigenere_cipher.vigenere_encrypt(plain_text, key)
    return jsonify({"encrypted_text": encrypted_text})

@app.route("/api/vigenere/decrypt", methods=["POST"])
def vigenere_decrypt_api():
    data = request.json

    if data is None or "cipher_text" not in data or "key" not in data:
        return jsonify({"error": "Missing cipher_text or key"}), 400

    cipher_text = data["cipher_text"]
    key = data["key"]

    if not isinstance(key, str) or key == "":
        return jsonify({"error": "Key must not be empty"}), 400

    if not key.isalpha():
        return jsonify({"error": "Key must contain only letters"}), 400

    decrypted_text = vigenere_cipher.vigenere_decrypt(cipher_text, key)
    return jsonify({"decrypted_text": decrypted_text})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Đường dẫn file: lab-02\ex02\requirements.txt

Flask>=2.3.2

Đường dẫn file: lab-02\ex02\cipher__init__.py

Đường dẫn file: lab-02\ex02\cipher\vigenere__init__.py

```
from .vigenere_cipher import VigenereCipher
```

Đường dẫn file: lab-02\ex02\cipher\vigenere\vigenere_cipher.py

```
class VigenereCipher:
    def __init__(self):
        pass

    def vigenere_encrypt(self, plain_text, key):
        encrypted_text = ""
        key_index = 0

        for char in plain_text:
            if char.isalpha():
                key_shift = ord(key[key_index % len(key)].upper()) - ord("A")

                if char.isupper():
                    encrypted_text += chr((ord(char) - ord("A") + key_shift) % 26
+ ord("A"))
                else:
                    encrypted_text += chr((ord(char) - ord("a") + key_shift) % 26
+ ord("a"))

                key_index += 1
            else:
                encrypted_text += char

        return encrypted_text

    def vigenere_decrypt(self, encrypted_text, key):
        decrypted_text = ""
```

```

        key_index = 0

        for char in encrypted_text:
            if char.isalpha():
                key_shift = ord(key[key_index % len(key)].upper()) - ord("A")

                if char.isupper():
                    decrypted_text += chr((ord(char) - ord("A") - key_shift) % 26
+ ord("A"))
                else:
                    decrypted_text += chr((ord(char) - ord("a") - key_shift) % 26
+ ord("a"))

                key_index += 1
            else:
                decrypted_text += char

        return decrypted_text

```

Đường dẫn file: `lab-02\ex03\api.py`

```

from flask import Flask, request, jsonify
from cipher.railfence import RailFenceCipher

app = Flask(__name__)
railfence_cipher = RailFenceCipher()

@app.route("/", methods=["GET"])
def index():
    return "Rail Fence API is running"

@app.route("/api/railfence/encrypt", methods=["POST"])
def railfence_encrypt():
    data = request.json

    if data is None or "plain_text" not in data or "key" not in data:
        return jsonify({"error": "Missing plain_text or key"}), 400

    plain_text = data["plain_text"]

    try:
        key = int(data["key"])
    except (ValueError, TypeError):
        return jsonify({"error": "Key must be an integer"}), 400

    encrypted_text = railfence_cipher.rail_fence_encrypt(plain_text, key)

```

```
    return jsonify({"encrypted_text": encrypted_text})

@app.route("/api/railfence/decrypt", methods=["POST"])
def railfence_decrypt():
    data = request.json

    if data is None or "cipher_text" not in data or "key" not in data:
        return jsonify({"error": "Missing cipher_text or key"}), 400

    cipher_text = data["cipher_text"]

    try:
        key = int(data["key"])
    except (ValueError, TypeError):
        return jsonify({"error": "Key must be an integer"}), 400

    decrypted_text = railfence_cipher.rail_fence_decrypt(cipher_text, key)
    return jsonify({"decrypted_text": decrypted_text})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

Đường dẫn file: lab-02\ex03\requirements.txt

```
Flask>=2.3.2
```

Đường dẫn file: lab-02\ex03\cipher__init__.py

Đường dẫn file: lab-02\ex03\cipher\railfence__init__.py

```
from .railfence_cipher import RailFenceCipher
```

Đường dẫn file: lab-02\ex03\cipher\railfence\railfence_cipher.py

```
class RailFenceCipher:
    def __init__(self):
        pass

    def rail_fence_encrypt(self, plain_text, num_rails):
        if num_rails <= 1 or plain_text == "":
            return plain_text

        rails = [[] for _ in range(num_rails)]
        rail_index = 0
        direction = 1

        for char in plain_text:
            rails[rail_index].append(char)

            if rail_index == 0:
                direction = 1
            if rail_index == num_rails - 1:
                direction = -1

            rail_index += direction

        cipher_text = "".join("".join(rail) for rail in rails)
        return cipher_text

    def rail_fence_decrypt(self, cipher_text, num_rails):
        if num_rails <= 1 or cipher_text == "":
            return cipher_text

        rail_lengths = [0] * num_rails
        rail_index = 0
        direction = 1

        for _ in range(len(cipher_text)):
            rail_lengths[rail_index] += 1

            if rail_index == 0:
                direction = 1
            if rail_index == num_rails - 1:
                direction = -1

            rail_index += direction

        rails = []
        start = 0

        for length in rail_lengths:
            rails.append(list(cipher_text[start:start + length]))
            start += length
```

```
plain_text = []
rail_index = 0
direction = 1

for _ in range(len(cipher_text)):
    plain_text.append(rails[rail_index].pop(0))

    if rail_index == 0:
        direction = 1
    if rail_index == num_rails - 1:
        direction = -1

    rail_index += direction

return "".join(plain_text)
```

Đường dẫn file: `lab-04\requirements.txt`

```
pycryptodome
cryptography
tornado
```

Đường dẫn file: `lab-04\aes_rsa_socket\client.py`

```
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import socket
import threading
import hashlib
import sys

# Initialize client socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 12345))

# Generate RSA key pair
client_key = RSA.generate(2048)

# Receive server's public key
server_public_key = RSA.import_key(client_socket.recv(2048))

# Send client's public key to the server
client_socket.send(client_key.publickey().export_key(format='PEM'))
```

```
# Receive encrypted AES key from the server
encrypted_aes_key = client_socket.recv(2048)

# Decrypt the AES key using client's private key
cipher_rsa = PKCS1_OAEP.new(client_key)
aes_key = cipher_rsa.decrypt(encrypted_aes_key)

# Function to encrypt message
def encrypt_message(key, message):
    cipher = AES.new(key, AES.MODE_CBC)
    ciphertext = cipher.encrypt(pad(message.encode(), AES.block_size))
    return cipher.iv + ciphertext

# Function to decrypt message
def decrypt_message(key, encrypted_message):
    iv = encrypted_message[:AES.block_size]
    ciphertext = encrypted_message[AES.block_size:]
    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted_message = unpad(cipher.decrypt(ciphertext), AES.block_size)
    return decrypted_message.decode()

# Function to receive messages from server
def receive_messages():
    while True:
        encrypted_message = client_socket.recv(1024)
        decrypted_message = decrypt_message(aes_key, encrypted_message)
        sys.stdout.write('\nReceived: ' + decrypted_message + '\n') # In tin nhắn
        đã nhận
        sys.stdout.write("Enter message ('exit' to quit): ") # In lại prompt nhập
        tin nhắn
        sys.stdout.flush()

# Start the receiving thread
receive_thread = threading.Thread(target=receive_messages)
receive_thread.start()

# Send messages to the server
while True:
    sys.stdout.write("Enter message ('exit' to quit): ")
    sys.stdout.flush() # Đảm bảo prompt được in ngay lập tức
    message = input() # Nhận đầu vào từ người dùng
    encrypted_message = encrypt_message(aes_key, message)
    client_socket.send(encrypted_message)
    if message == "exit":
        break

# Close the connection when done
client_socket.close()
```

Đường dẫn file: lab-04\aes_rsa_socket\client2.py

```

from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import socket
import threading
import hashlib
import sys

# Initialize client socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(('localhost', 12345))

# Generate RSA key pair
client_key = RSA.generate(2048)

# Receive server's public key
server_public_key = RSA.import_key(client_socket.recv(2048))

# Send client's public key to the server
client_socket.send(client_key.publickey().export_key(format='PEM'))

# Receive encrypted AES key from the server
encrypted_aes_key = client_socket.recv(2048)

# Decrypt the AES key using client's private key
cipher_rsa = PKCS1_OAEP.new(client_key)
aes_key = cipher_rsa.decrypt(encrypted_aes_key)

# Function to encrypt message
def encrypt_message(key, message):
    cipher = AES.new(key, AES.MODE_CBC)
    ciphertext = cipher.encrypt(pad(message.encode(), AES.block_size))
    return cipher.iv + ciphertext

# Function to decrypt message
def decrypt_message(key, encrypted_message):
    iv = encrypted_message[:AES.block_size]
    ciphertext = encrypted_message[AES.block_size:]
    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted_message = unpad(cipher.decrypt(ciphertext), AES.block_size)
    return decrypted_message.decode()

# Function to receive messages from server
def receive_messages():
    while True:
        encrypted_message = client_socket.recv(1024)
        decrypted_message = decrypt_message(aes_key, encrypted_message)
        sys.stdout.write('\nReceived: ' + decrypted_message + '\n') # In tin nhắn
        đã nhận

```

```
        sys.stdout.write("Enter message ('exit' to quit): ") # In lại prompt nhập
        tin nhả
        sys.stdout.flush()

# Start the receiving thread
receive_thread = threading.Thread(target=receive_messages)
receive_thread.start()

# Send messages to the server
while True:
    sys.stdout.write("Enter message ('exit' to quit): ")
    sys.stdout.flush() # Đảm bảo prompt được in ngay lập tức
    message = input() # Nhận đầu vào từ người dùng
    encrypted_message = encrypt_message(aes_key, message)
    client_socket.send(encrypted_message)
    if message == "exit":
        break

# Close the connection when done
client_socket.close()
```

Đường dẫn file: `lab-04\aes_rsa_socket\requirements.txt`

```
pycryptodome
```

Đường dẫn file: `lab-04\aes_rsa_socket\server.py`

```
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import socket
import threading
import hashlib

# Initialize server socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_socket.bind(('localhost', 12345))
server_socket.listen(5)

# Generate RSA key pair
server_key = RSA.generate(2048)

# List of connected clients
clients = []
```

```
# Function to encrypt message
def encrypt_message(key, message):
    cipher = AES.new(key, AES.MODE_CBC)
    ciphertext = cipher.encrypt(pad(message.encode(), AES.block_size))
    return cipher.iv + ciphertext

# Function to decrypt message
def decrypt_message(key, encrypted_message):
    iv = encrypted_message[:AES.block_size]
    ciphertext = encrypted_message[AES.block_size:]
    cipher = AES.new(key, AES.MODE_CBC, iv)
    decrypted_message = unpad(cipher.decrypt(ciphertext), AES.block_size)
    return decrypted_message.decode()

# Function to handle client connection
def handle_client(client_socket, client_address):
    print(f"Connected with {client_address}")

    # Send server's public key to client
    client_socket.send(server_key.publickey().export_key(format='PEM'))

    # Receive client's public key
    client_received_key = RSA.import_key(client_socket.recv(2048))

    # Generate AES key for message encryption
    aes_key = get_random_bytes(16)

    # Encrypt the AES key using the client's public key
    cipher_rsa = PKCS1_OAEP.new(client_received_key)
    encrypted_aes_key = cipher_rsa.encrypt(aes_key)
    client_socket.send(encrypted_aes_key)

    # Add client to the list
    clients.append((client_socket, aes_key))

    while True:
        encrypted_message = client_socket.recv(1024)
        decrypted_message = decrypt_message(aes_key, encrypted_message)
        print(f"Received from {client_address}: {decrypted_message}")

        # Send received message to all other clients
        for client, key in clients:
            if client != client_socket:
                encrypted = encrypt_message(key, decrypted_message)
                client.send(encrypted)

        if decrypted_message == "exit":
            break

    clients.remove((client_socket, aes_key))
    client_socket.close()
    print(f"Connection with {client_address} closed")
```

```
# Accept and handle client connections
while True:
    client_socket, client_address = server_socket.accept()
    client_thread = threading.Thread(target=handle_client, args=(client_socket,
client_address))
    client_thread.start()
```

Đường dẫn file: `lab-04\dh_key_pair\client.py`

```
from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC
from cryptography.hazmat.primitives import hashes

def generate_client_key_pair(parameters):
    private_key = parameters.generate_private_key()
    public_key = private_key.public_key()
    return private_key, public_key

def derive_shared_secret(private_key, server_public_key):
    shared_key = private_key.exchange(server_public_key)
    return shared_key

def main():
    # Load server's public key
    with open("server_public_key.pem", "rb") as f:
        server_public_key = serialization.load_pem_public_key(f.read())

    parameters = server_public_key.parameters()
    private_key, public_key = generate_client_key_pair(parameters)

    shared_secret = derive_shared_secret(private_key, server_public_key)

    print("Shared Secret:", shared_secret.hex())

if __name__ == "__main__":
    main()
```

Đường dẫn file: `lab-04\dh_key_pair\requirements.txt`

```
cryptography
```

Đường dẫn file: lab-04\dh_key_pair\server.py

```
from cryptography.hazmat.primitives.asymmetric import dh
from cryptography.hazmat.primitives import serialization

def generate_dh_parameters():
    parameters = dh.generate_parameters(generator=2, key_size=2048)
    return parameters

def generate_server_key_pair(parameters):
    private_key = parameters.generate_private_key()
    public_key = private_key.public_key()
    return private_key, public_key

def main():
    parameters = generate_dh_parameters()
    private_key, public_key = generate_server_key_pair(parameters)

    with open("server_public_key.pem", "wb") as f:
        f.write(public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
        ))

if __name__ == "__main__":
    main()
```

Đường dẫn file: lab-04\hash\blake2_hash.py

```
import hashlib

def blake2(message):
    blake2_hash = hashlib.blake2b(digest_size=64)
    blake2_hash.update(message)
    return blake2_hash.digest()

def main():
    text = input("Nhập chuỗi văn bản: ").encode('utf-8')
    hashed_text = blake2(text)

    print("Chuỗi văn bản đã nhập:", text.decode('utf-8'))
    print("BLAKE2 Hash:", hashed_text.hex())

if __name__ == "__main__":
    main()
```

Đường dẫn file: lab-04\hash\md5_hash.py

```
# Hàm dịch vòng trái (left rotate) cho 1 giá trị 32-bit
def left_rotate(value, shift):
    return ((value << shift) | (value >> (32 - shift))) & 0xFFFFFFFF

# Hàm băm MD5 chính
def md5(message):
    # Khởi tạo các biến ban đầu theo chuẩn MD5
    a = 0x67452301
    b = 0xEFCDAB89
    c = 0x98BADCFE
    d = 0x10325476

    # Tiền xử lý chuỗi văn bản (padding)
    original_length = len(message)
    message += b'\x80' # Thêm bit '1' vào cuối chuỗi
    while len(message) % 64 != 56: # Đệm thêm các bit '0' sao cho độ dài mod 64
        message += b'\x00'
    message += original_length.to_bytes(8, 'little') # Thêm độ dài ban đầu (64-bit little endian)

    # Chia chuỗi thành các khối 512-bit (64 bytes)
    for i in range(0, len(message), 64):
        block = message[i:i+64]

        # Tách block thành 16 từ 32-bit (little endian)
        words = [int.from_bytes(block[j:j+4], 'little') for j in range(0, 64, 4)]

        # Lưu giá trị ban đầu
        a0, b0, c0, d0 = a, b, c, d

        # Vòng lặp chính của thuật toán MD5 gồm 64 bước
        for j in range(64):
            if j < 16:
                f = (b & c) | ((~b) & d)
                g = j
            elif j < 32:
                f = (d & b) | ((~d) & c)
                g = (5 * j + 1) % 16
            elif j < 48:
                f = b ^ c ^ d
                g = (3 * j + 5) % 16
            else:
                f = c ^ (b | (~d))
                g = (7 * j) % 16

            temp = d
            d = c
            c = b
```

```
# Thực hiện các phép tính chính: cộng, dịch vòng trái, cộng thêm giá
trị trong words[g]
b = (b + left_rotate((a + f + 0x5A827999 + words[g]) & 0xFFFFFFFF, 3))
& 0xFFFFFFFF
a = temp

# Cộng dồn vào giá trị ban đầu
a = (a + a0) & 0xFFFFFFFF
b = (b + b0) & 0xFFFFFFFF
c = (c + c0) & 0xFFFFFFFF
d = (d + d0) & 0xFFFFFFFF

# Trả về chuỗi băm ở dạng hex
return '{:08x}{:08x}{:08x}{:08x}'.format(a, b, c, d)

# Nhập dữ liệu từ người dùng
input_string = input("Nhập chuỗi cần băm: ")
md5_hash = md5(input_string.encode('utf-8'))

# In kết quả
print("Mã băm MD5 của chuỗi '{}' là: {}".format(input_string, md5_hash))
```

Đường dẫn file: `lab-04\hash\md5_hash_2.py`

```
import math
# Hàm dịch vòng trái (left rotate) cho 1 giá trị 32-bit
def left_rotate(value, shift):
    return ((value << shift) | (value >> (32 - shift))) & 0xFFFFFFFF

# Bảng hằng số K dùng trong MD5 (64 phần tử)
K = [int((2**32) * abs(math.sin(i + 1))) & 0xFFFFFFFF for i in range(64)]

# Bảng số lượng dịch trái (shift amounts)
s = [
    7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22, 7, 12, 17, 22,
    5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20, 5, 9, 14, 20,
    4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23, 4, 11, 16, 23,
    6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21, 6, 10, 15, 21
]

# Hàm băm MD5 chính
def md5(message):
    # Khởi tạo các giá trị ban đầu (IV)
    a0 = 0x67452301
    b0 = 0xefcdab89
    c0 = 0x98badcfe
    d0 = 0x10325476
```

```
# Tiền xử lý chuỗi (Padding)
original_length = len(message) * 8 # Tính độ dài bit
message += b'\x80'
while (len(message) % 64) != 56:
    message += b'\x00'
message += original_length.to_bytes(8, byteorder='little') # Thêm độ dài
(little endian)

# Xử lý các khối 512-bit (64 byte)
for offset in range(0, len(message), 64):
    block = message[offset:offset + 64]
    M = [int.from_bytes(block[i:i + 4], byteorder='little') for i in range(0,
64, 4)]

    A, B, C, D = a0, b0, c0, d0 # Sao chép giá trị ban đầu

    # Vòng lặp chính (64 bước)
    for i in range(64):
        if i < 16:
            F = (B & C) | (~B & D)
            g = i
        elif i < 32:
            F = (D & B) | (~D & C)
            g = (5 * i + 1) % 16
        elif i < 48:
            F = B ^ C ^ D
            g = (3 * i + 5) % 16
        else:
            F = C ^ (B | ~D)
            g = (7 * i) % 16

        temp = (A + F + K[i] + M[g]) & 0xFFFFFFFF
        A, D, C, B = D, C, B, (B + left_rotate(temp, s[i])) & 0xFFFFFFFF

    # Cộng dồn vào kết quả
    a0 = (a0 + A) & 0xFFFFFFFF
    b0 = (b0 + B) & 0xFFFFFFFF
    c0 = (c0 + C) & 0xFFFFFFFF
    d0 = (d0 + D) & 0xFFFFFFFF

# Trả về kết quả dạng hex (little endian)
digest = sum(x << (32 * i) for i, x in enumerate([a0, b0, c0, d0]))
return ''.join(f'{digest >> (8 * i) & 0xFF:02x}' for i in range(16))

# ===== Chạy thử =====
input_string = input("Nhập chuỗi cần băm: ")
md5_hash = md5(input_string.encode('utf-8'))

print("Mã băm MD5 của chuỗi '{}' là: {}".format(input_string, md5_hash))
```

Đường dẫn file: lab-04\hash\md5_library.py

```
import hashlib

def calculate_md5(input_string):
    md5_hash = hashlib.md5()
    md5_hash.update(input_string.encode('utf-8'))
    return md5_hash.hexdigest()

input_string = input("Nhập chuỗi cần băm: ")

md5_hash = calculate_md5(input_string)

# In ra kết quả mã băm
print("Mã băm MD5 của chuỗi '{}' là: {}".format(input_string, md5_hash))
```

Đường dẫn file: lab-04\hash\sha256_hash.py

```
import hashlib

def calculate_sha256_hash(data):
    sha256_hash = hashlib.sha256()
    sha256_hash.update(data.encode('utf-8'))
    return sha256_hash.hexdigest()

data_to_hash = input("Nhập dữ liệu để hash bằng SHA-256: ")
hash_value = calculate_sha256_hash(data_to_hash)
print("Giá trị hash SHA-256:", hash_value)
```

Đường dẫn file: lab-04\hash\sha3_hash.py

```
from Crypto.Hash import SHA3_256

def sha3(message):
    sha3_hash = SHA3_256.new()
    sha3_hash.update(message)
    return sha3_hash.digest()

def main():
    text = input("Nhập chuỗi văn bản: ").encode('utf-8')
    hashed_text = sha3(text)

    print("Chuỗi văn bản đã nhập:", text.decode('utf-8'))
```

```
print("SHA-3 Hash:", hashed_text.hex())

if __name__ == "__main__":
    main()
```

Đường dẫn file: lab-04\websocket\client.py

```
import tornado.ioloop
import tornado.websocket

class WebSocketClient:
    def __init__(self, io_loop):
        self.connection = None
        self.io_loop = io_loop

    def start(self):
        self.connect_and_read()

    def stop(self):
        self.io_loop.stop()

    def connect_and_read(self):
        print("Reading...")
        tornado.websocket.websocket_connect(
            url="ws://localhost:8888/websocket/",
            callback=self.maybe_retry_connection,
            on_message_callback=self.on_message,
            ping_interval=30,
            ping_timeout=10,
        )

    def maybe_retry_connection(self, future) -> None:
        try:
            self.connection = future.result()
        except:
            print('Could not reconnect, retrying in 3 seconds...')
            self.io_loop.call_later(3, self.connect_and_read)
            return

        self.connection.read_message(callback=self.on_message)

    def on_message(self, message):
        if message is None:
            print('Disconnected, reconnecting...')
            self.connect_and_read()
            return

        print(f"Received word from server: {message}")
```

```

        self.connection.read_message(callback=self.on_message)

def main():
    io_loop = tornado.ioloop.IOLoop.current()

    client = WebSocketClient(io_loop)
    io_loop.add_callback(client.start)

    io_loop.start()

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-04\websocket\client2.py`

```

import tornado.ioloop
import tornado.websocket

class WebSocketClient:
    def __init__(self, io_loop):
        self.connection = None
        self.io_loop = io_loop

    def start(self):
        self.connect_and_read()

    def stop(self):
        self.io_loop.stop()

    def connect_and_read(self):
        print("Reading...")
        tornado.websocket.websocket_connect(
            url="ws://localhost:8888/websocket/",
            callback=self.maybe_retry_connection,
            on_message_callback=self.on_message,
            ping_interval=30,
            ping_timeout=10,
        )

    def maybe_retry_connection(self, future) -> None:
        try:
            self.connection = future.result()
        except:
            print('Could not reconnect, retrying in 3 seconds...')
            self.io_loop.call_later(3, self.connect_and_read)
            return

        self.connection.read_message(callback=self.on_message)

```

```
def on_message(self, message):
    if message is None:
        print('Disconnected, reconnecting...')
        self.connect_and_read()
        return

    print(f"Received word from server: {message}")

    self.connection.read_message(callback=self.on_message)

def main():
    io_loop = tornado.ioloop.IOLoop.current()

    client = WebSocketClient(io_loop)
    io_loop.add_callback(client.start)

    io_loop.start()

if __name__ == "__main__":
    main()
```

Đường dẫn file: `lab-04\websocket\requirements.txt`

```
tornado
```

Đường dẫn file: `lab-04\websocket\server.py`

```
import random
import tornado.web
import tornado.ioloop
import tornado.websocket

class WebSocketServer(tornado.websocket.WebSocketHandler):
    clients = set()

    def open(self):
        WebSocketServer.clients.add(self)

    def on_close(self):
        WebSocketServer.clients.remove(self)

    @classmethod
    def send_message(cls, message: str):
        print(f"Sending message to {len(cls.clients)} client(s).")
        for client in cls.clients:
```

```

        client.write_message(message)

class RandomWordSelector:
    def __init__(self, word_list):
        self.word_list = word_list

    def sample(self):
        return random.choice(self.word_list)

def main():
    app = tornado.web.Application([
        (r"/websocket/", WebSocketServer),
    ], websocket_ping_interval=30,
        websocket_ping_timeout=10)

    app.listen(8888)

    io_loop = tornado.ioloop.IOLoop.current()

    word_selector = RandomWordSelector(['apple', 'banana', 'orange', 'grape',
    'melon'])

    periodic_callback = tornado.ioloop.PeriodicCallback(
        lambda: WebSocketServer.send_message(word_selector.sample()), 3000
    )
    periodic_callback.start()

    io_loop.start()

if __name__ == "__main__":
    main()

```

Đường dẫn file: [lab-05\base64\decode.py](#)

```

import base64
import binascii

def validate_base64(s):
    """Kiểm tra sơ bộ tính hợp lệ của chuỗi Base64"""
    # Loại bỏ các ký tự khoảng trắng nếu có
    s = s.strip()

    # Độ dài chuỗi Base64 phải chia hết cho 4 (kể cả phần bù = nếu có)
    if len(s) % 4 != 0:
        return False, "Độ dài chuỗi Base64 không hợp lệ (phải chia hết cho 4)."

    # Tập ký tự hợp lệ trong Base64
    valid_chars =
    set("ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/=")

```

```

if not all(char in valid_chars for char in s):
    return False, "Chuỗi chứa ký tự không hợp lệ trong bảng mã Base64."

return True, ""

def main():
    print("=== BASE64 DECODE ===")
    try:
        user_input = input("Nhập chuỗi Base64 cần giải mã: ").strip()

        # Kiểm tra tính hợp lệ trước khi giải mã
        is_valid, err_msg = validate_base64(user_input)
        if not is_valid:
            print(f"\n[Lỗi Định Dạng] {err_msg}")
            return

        # Giải mã Base64
        # Chuyển chuỗi đầu vào thành bytes
        encoded_bytes = user_input.encode('utf-8')

        # Giải mã
        decoded_bytes = base64.b64decode(encoded_bytes)

        # Chuyển bytes kết quả thành chuỗi utf-8
        decoded_str = decoded_bytes.decode('utf-8')

        print("\n--- Kết quả giải mã ---")
        print(f"Chuỗi Base64: {user_input}")
        print(f"Chuỗi ban đầu: {decoded_str}")
        print("-----")

    except (binascii.Error, ValueError) as e:
        print(f"\n[Lỗi Giải Mã] Chuỗi Base64 không hợp lệ hoặc sai định dạng padding: {e}")
    except UnicodeDecodeError:
        print(f"\n[Lỗi Giải Mã] Chuỗi giải mã thành công dạng bytes nhưng không thể hiển thị dưới dạng văn bản UTF-8 (có thể đây là dữ liệu nhị phân).")
    except Exception as e:
        print(f"\n[Lỗi] Có lỗi hệ thống xảy ra: {e}")

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-05\base64\encode.py`

```

import base64

def main():
    print("=== BASE64 ENCODE ===")

```

```

try:
    user_input = input("Nhập chuỗi văn bản cần mã hóa: ")

    # Chuyển chuỗi thành bytes
    input_bytes = user_input.encode('utf-8')

    # Mã hóa Base64
    encoded_bytes = base64.b64encode(input_bytes)

    # Chuyển bytes kết quả thành chuỗi để hiển thị
    encoded_str = encoded_bytes.decode('utf-8')

    print("\n--- Kết quả mã hóa ---")
    print(f"Chuỗi ban đầu: {user_input}")
    print(f"Chuỗi Base64: {encoded_str}")
    print("-----")

except Exception as e:
    print(f"Có lỗi xảy ra: {e}")

if __name__ == "__main__":
    main()

```

Đường dẫn file: `lab-05\blockchain\block.py`

```

import hashlib
import json

class Block:
    def __init__(self, index, timestamp, transactions, proof, previous_hash):
        self.index = index
        self.timestamp = timestamp
        self.transactions = transactions
        self.proof = proof
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        """Tính toán mã băm SHA-256 của Block hiện tại"""
        # Sắp xếp các transaction theo key để đảm bảo tính nhất quán của chuỗi
        JSON
        block_string = json.dumps({
            'index': self.index,
            'timestamp': self.timestamp,
            'transactions': self.transactions,
            'proof': self.proof,
            'previous_hash': self.previous_hash
        }, sort_keys=True).encode('utf-8')

```

```

        return hashlib.sha256(block_string).hexdigest()

    def to_dict(self):
        """Chuyển đổi block thành dictionary phục vụ in ấn"""
        return {
            'index': self.index,
            'timestamp': self.timestamp,
            'transactions': self.transactions,
            'proof': self.proof,
            'previous_hash': self.previous_hash,
            'hash': self.hash
        }

```

Đường dẫn file: `lab-05\blockchain\blockchain.py`

```

import time
import hashlib
from block import Block

class Blockchain:
    def __init__(self):
        self.chain = []
        self.current_transactions = []
        self.difficulty = 4 # Số chữ số 0 đứng đầu yêu cầu khi đào block (độ khó)

        # Tạo Genesis Block (Block đầu tiên của chuỗi)
        self.new_block(proof=100, previous_hash='1')

    def new_block(self, proof, previous_hash=None):
        """Tạo một Block mới và thêm vào chuỗi"""
        block = Block(
            index=len(self.chain) + 1,
            timestamp=time.time(),
            transactions=self.current_transactions,
            proof=proof,
            previous_hash=previous_hash or self.chain[-1].hash
        )

        # Reset danh sách các giao dịch hiện tại
        self.current_transactions = []

        # Thêm block vào chuỗi
        self.chain.append(block)
        return block

    def new_transaction(self, sender, recipient, amount):
        """Thêm giao dịch mới vào danh sách giao dịch chờ đào"""
        self.current_transactions.append({
            'sender': sender,

```

```

        'recipient': recipient,
        'amount': amount,
        'timestamp': time.time()
    })
    return self.last_block.index + 1

@property
def last_block(self):
    """Lấy Block cuối cùng trong chuỗi"""
    return self.chain[-1]

def proof_of_work(self, last_block):
    """
    Thuật toán Proof of Work (Mô tả quá trình đào):
    Tìm một số proof sao cho hash(last_proof, proof, last_hash) có đúng số số
    0 đứng đầu bằng độ khó.
    """
    last_proof = last_block.proof
    last_hash = last_block.hash
    proof = 0

    while self.valid_proof(last_proof, proof, last_hash) is False:
        proof += 1

    return proof

def valid_proof(self, last_proof, proof, last_hash):
    """Kiểm tra xem mã băm có thỏa mãn độ khó hay không"""
    guess = f'{last_proof}{proof}{last_hash}'.encode('utf-8')
    guess_hash = hashlib.sha256(guess).hexdigest()
    return guess_hash[:self.difficulty] == '0' * self.difficulty

def is_chain_valid(self):
    """Kiểm tra tính hợp lệ toàn bộ blockchain"""
    last_block = self.chain[0]
    current_index = 1

    while current_index < len(self.chain):
        block = self.chain[current_index]

        # 1. Kiểm tra Previous Hash có khớp với Hash thực tế của block trước
        đó không
        if block.previous_hash != last_block.hash:
            print(f"[-] Lỗi tính toán vẹn: Block {block.index} có
previous_hash không khớp với hash của Block {last_block.index}")
            return False

        # 2. Kiểm tra Proof of Work có hợp lệ không
        if not self.valid_proof(last_block.proof, block.proof,
last_block.hash):
            print(f"[-] Lỗi Proof of Work: Block {block.index} có proof
{block.proof} không hợp lệ")
            return False

```

```

        # 3. Kiểm tra tính đúng đắn của băm nội tại block
        if block.hash != block.calculate_hash():
            print(f"[-] Lỗi băm nội tại: Block {block.index} có dữ liệu bị
thay đổi sau khi tạo")
            return False

        last_block = block
        current_index += 1

    return True

```

Đường dẫn file: `lab-05\blockchain\test_blockchain.py`

```

import json
from blockchain import Blockchain

def print_block(block):
    print(f"\n===== Block {block.index} =====")
    print(f"Timestamp      : {block.timestamp}")
    print(f"Previous Hash: {block.previous_hash}")
    print(f"Proof           : {block.proof}")
    print(f"Hash            : {block.hash}")
    print("Transactions :")
    if len(block.transactions) == 0:
        print("    (Không có giao dịch - Genesis Block)")
    for tx in block.transactions:
        print(f"    - {tx['sender']} -> {tx['recipient']}: {tx['amount']} coins
({tx['timestamp']})")
    print("=====")

def main():
    print("=== KHIẾM THỬ MÔ PHÒNG BLOCKCHAIN ===")

    # 1. Khởi tạo Blockchain
    print("\nKhởi tạo Blockchain mới...")
    blockchain = Blockchain()

    # In Block đầu tiên (Genesis Block)
    print_block(blockchain.chain[0])

    # 2. Thêm giao dịch và Đào Block 1
    print("\n[+] Thêm giao dịch mới vào Block 1...")
    blockchain.new_transaction(sender="Lê Viết Đức", recipient="Nguyễn Văn A",
amount=50.5)
    blockchain.new_transaction(sender="Nguyễn Văn A", recipient="Trần Thị B",
amount=10.0)

    print("Đang chạy Proof of Work để tìm Hash hợp lệ (Đang đào)...")
    last_block = blockchain.last_block

```

```

proof = blockchain.proof_of_work(last_block)
print(f"-> Tìm thấy Proof: {proof}")

# Tạo Block 1
block1 = blockchain.new_block(proof)
print("Đào thành công Block 1!")
print_block(block1)

# 3. Thêm giao dịch và Đào Block 2
print("\n[+] Thêm giao dịch mới vào Block 2...")
blockchain.new_transaction(sender="Trần Thị B", recipient="Lê Viết Đức",
amount=5.0)

print("Đang chạy Proof of Work (Đang đào)...")
last_block = blockchain.last_block
proof = blockchain.proof_of_work(last_block)
print(f"-> Tìm thấy Proof: {proof}")

# Tạo Block 2
block2 = blockchain.new_block(proof)
print("Đào thành công Block 2!")
print_block(block2)

# 4. Kiểm tra tính hợp lệ của Blockchain
print("\n--- Kiểm tra tính hợp lệ của Chuỗi Blockchain ---")
is_valid = blockchain.is_chain_valid()
print(f"Is Blockchain Valid: {is_valid}")
print("-----")

# 5. Thử nghiệm thay đổi dữ liệu trái phép (Tấn công Blockchain)
print("\n[Tấn Công Giả Lập] Thay đổi số tiền giao dịch tại Block 1 từ 50.5
thành 999.0...")
blockchain.chain[1].transactions[0]['amount'] = 999.0

# Tính lại hash xem hệ thống phát hiện không
print("--- Kiểm tra tính hợp lệ sau khi bị thay đổi dữ liệu ---")
is_valid_after_attack = blockchain.is_chain_valid()
print(f"Is Blockchain Valid: {is_valid_after_attack}")
print("-----")

if not is_valid_after_attack:
    print("[SUCCESS] Hệ thống phát hiện dữ liệu Block 1 đã bị giả mạo và từ
chối chuỗi này!")
else:
    print("[LỖI] Hệ thống không phát hiện ra việc giả mạo dữ liệu!")

if __name__ == "__main__":
    main()

```

Đường dẫn file: [lab-05\img-hidden\decode_image.py](#)

```

from PIL import Image
import os

def decode_message_from_image(image_path):
    print(f"Đang đọc ảnh để giải mã từ: {image_path}...")
    if not os.path.exists(image_path):
        print(f"[LỖI] Không tìm thấy ảnh tại: {image_path}")
        return None

    img = Image.open(image_path)
    if img.mode != 'RGB':
        img = img.convert('RGB')

    pixels = list(img.getdata())

    bin_data = ""
    decoded_chars = []

    print("Đang tiến hành trích xuất các bit LSB...")

    for pixel in pixels:
        r, g, b = pixel

        # Đọc LSB từ R, G, B
        bin_data += str(r & 1)
        bin_data += str(g & 1)
        bin_data += str(b & 1)

        # Gom đủ 8 bits (1 byte) để chuyển đổi thành ký tự
        while len(bin_data) >= 8:
            byte_str = bin_data[:8]
            bin_data = bin_data[8:]

            byte_val = int(byte_str, 2)

            # Nếu gặp ký tự Null (\x00) thì dừng việc giải mã vì đây là dấu hiệu
            kết thúc
            if byte_val == 0:
                return "".join(decoded_chars)

            try:
                decoded_chars.append(chr(byte_val))
            except Exception:
                # Bỏ qua nếu có ký tự không thể giải mã
                pass

        # Nếu đi hết cả ảnh mà không gặp ký tự Null
        return "".join(decoded_chars)

def main():
    print("=== GIẤU TIN TRONG ẢNH (DECODE) ===")

```

```

current_dir = os.path.dirname(os.path.abspath(__file__))
encoded_path = os.path.join(current_dir, "encoded_image.png")
original_path = os.path.join(current_dir, "input_image.png")

print("Vui lòng lựa chọn ảnh để tiến hành trích xuất tin:")
print("1. Giải mã từ ảnh đã giấu tin (encoded_image.png)")
print("2. Giải mã từ ảnh gốc (input_image.png) để kiểm tra")

choice = input("Lựa chọn (1 hoặc 2): ").strip()

if choice == '2':
    target_path = original_path
    is_original = True
else:
    target_path = encoded_path
    is_original = False

decoded_msg = decode_message_from_image(target_path)

if decoded_msg is not None:
    print("\n--- Kết quả trích xuất ---")
    if is_original:
        print(f"Thông điệp trích xuất từ ảnh gốc: {decoded_msg[:100]}... (Rác dữ liệu)")
        print("\n[GIẢI THÍCH] Tại sao không lấy được thông điệp từ ảnh gốc?")
        print("- Trong ảnh gốc (input_image.png), các bit ít quan trọng nhất (LSB) chứa các chi tiết màu sắc tự nhiên của ảnh.")
        print("- Khi giải mã các bit này, chúng ta chỉ nhận được các giá trị ngẫu nhiên (rác dữ liệu) thay vì thông điệp bí mật.")
        print("- Chỉ khi ảnh đã qua quá trình giấu tin (encoded_image.png), các bit LSB mới được sắp xếp có chủ đích để tạo thành các ký tự của thông điệp.")
    else:
        print(f"Thông điệp bí mật trích xuất được: {decoded_msg}")
        print("-----")

if __name__ == "__main__":
    main()

```

Đường dẫn file: lab-05\img-hidden\encode_image.py

```

from PIL import Image
import os

def msg_to_bin(msg):
    """Chuyển đổi chuỗi thông điệp thành chuỗi bit nhị phân (có ký tự Null \x00 kết thúc)"""
    bin_data = []
    for char in msg:
        bin_data.append(format(ord(char), '08b'))

```

```

    bin_data.append('00000000') # Ký tự phân tách Null dùng để đánh dấu kết thúc
    thông điệp khi giải mã
    return ''.join(bin_data)

def encode_message_in_image(image_path, message, output_path):
    print(f"Đang đọc ảnh gốc từ: {image_path}...")
    if not os.path.exists(image_path):
        print(f"[LỖI] Không tìm thấy ảnh đầu vào tại: {image_path}")
        return False

    img = Image.open(image_path)
    # Đảm bảo ảnh ở định dạng RGB
    if img.mode != 'RGB':
        img = img.convert('RGB')

    pixels = list(img.getdata())
    width, height = img.size

    # Chuyển thông điệp thành nhị phân
    bin_message = msg_to_bin(message)
    message_len = len(bin_message)

    # Kiểm tra xem ảnh có đủ dung lượng chứa thông điệp không
    # Mỗi pixel có 3 kênh màu (R, G, B), chứa được tối đa 3 bits
    capacity = len(pixels) * 3
    if message_len > capacity:
        print(f"[LỖI] Thông điệp quá dài ({message_len} bits). Ảnh chỉ chứa được
tối đa {capacity} bits.")
        return False

    print(f"Dung lượng ảnh: {capacity} bits. Kích thước thông điệp cần giấu:
{message_len} bits.")
    print("Đang tiến hành giấu tin vào các bit ít quan trọng nhất (LSB)...")

    new_pixels = []
    bit_idx = 0

    for pixel in pixels:
        r, g, b = pixel

        # Giấu bit vào kênh Red
        if bit_idx < message_len:
            r = (r & ~1) | int(bin_message[bit_idx])
            bit_idx += 1

        # Giấu bit vào kênh Green
        if bit_idx < message_len:
            g = (g & ~1) | int(bin_message[bit_idx])
            bit_idx += 1

        # Giấu bit vào kênh Blue
        if bit_idx < message_len:
            b = (b & ~1) | int(bin_message[bit_idx])
            bit_idx += 1

```

```

        new_pixels.append((r, g, b))

# Tạo ảnh mới với dữ liệu pixel đã sửa đổi và lưu lại
new_img = Image.new(img.mode, img.size)
new_img.putdata(new_pixels)
new_img.save(output_path)
print(f"[SUCCESS] Đã giấu tin thành công và lưu ảnh mới tại: {output_path}")
return True

def main():
    print("=== GIẤU TIN TRONG ẢNH (ENCODE) ===")

    # Thiết lập đường dẫn mặc định
    current_dir = os.path.dirname(os.path.abspath(__file__))
    input_path = os.path.join(current_dir, "input_image.png")
    output_path = os.path.join(current_dir, "encoded_image.png")

    message = input("Nhập thông điệp bí mật muốn giấu vào ảnh: ")
    if not message:
        message = "Day la thong diep mat tu Le Viet Duc 0533"
        print(f"Sử dụng thông điệp mặc định: '{message}'")

    encode_message_in_image(input_path, message, output_path)

if __name__ == "__main__":
    main()

```

Đường dẫn file: **lab-05\ssl\client.py**

```

import socket
import ssl
import sys

def main():
    host = '127.0.0.1'
    port = 8443

    print("=== SSL SOCKET CLIENT ===")

    # Tạo SSL Context cho Client
    # Sử dụng Purpose.SERVER_AUTH để xác thực server
    context = ssl.create_default_context(ssl.Purpose.SERVER_AUTH)

    # Bỏ qua xác thực chứng chỉ vì chúng ta dùng chứng chỉ tự ký (Self-signed) cho mục đích thực hành
    context.check_hostname = False
    context.verify_mode = ssl.CERT_NONE

```

```
# Tạo socket TCP kết nối đến server
raw_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    # Bọc socket TCP thường thành socket bảo mật SSL/TLS
    secure_socket = context.wrap_socket(raw_socket, server_hostname=host)

    print(f"Đang kết nối bảo mật tới server tại {host}:{port}...")
    secure_socket.connect((host, port))
    print("[+] Kết nối và bắt tay SSL/TLS thành công!")

    # Nhập tin nhắn gửi lên server
    message = input("Nhập tin nhắn để gửi đến Server: ")
    if not message:
        message = "Tin nhắn mặc định từ client"

    # Gửi dữ liệu đã được mã hóa
    secure_socket.sendall(message.encode('utf-8'))
    print("[*] Tin nhắn đã được mã hóa và gửi đi.")

    # Nhận phản hồi bảo mật từ server
    response = secure_socket.recv(1024)
    print(f"\n[Server phản hồi]: {response.decode('utf-8')}")

    # Đóng kết nối
    secure_socket.close()
    print("\n[-] Kết nối an toàn đã được đóng.")

except ConnectionRefusedError:
    print("\n[LỖI] Không thể kết nối tới Server. Hãy đảm bảo Server đang chạy trước.")
except Exception as e:
    print(f"\n[LỖI] Có lỗi xảy ra: {e}")

if __name__ == "__main__":
    main()
```

Đường dẫn file: **lab-05\ssl\server.py**

```
import socket
import ssl
import os

def main():
    # Cấu hình địa chỉ và cổng
    host = '127.0.0.1'
    port = 8443

    # Lấy đường dẫn tuyệt đối tới file chứng chỉ và khóa riêng
```

```

current_dir = os.path.dirname(os.path.abspath(__file__))
cert_path = os.path.join(current_dir, 'certificates', 'server.crt')
key_path = os.path.join(current_dir, 'certificates', 'server.key')

if not os.path.exists(cert_path) or not os.path.exists(key_path):
    print(f"[LỖI] Không tìm thấy file chứng chỉ hoặc khóa riêng.")
    print(f"Vui lòng chạy file certificates/generate_cert.bat trước.")
    return

print("=== SSL SOCKET SERVER ===")

# Tạo SSL Context cho Server
context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
context.load_cert_chain(certfile=cert_path, keyfile=key_path)

# Tạo socket TCP cơ bản
bind_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
bind_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
bind_socket.bind((host, port))
bind_socket.listen(5)

print(f"Server đang lắng nghe kết nối bảo mật tại {host}:{port}...")

try:
    while True:
        # Chấp nhận kết nối TCP từ client
        client_socket, from_address = bind_socket.accept()
        print(f"\n[+] Có kết nối TCP mới từ: {from_address}")

        try:
            # Thiết lập bắt tay SSL/TLS (wrap socket)
            secure_socket = context.wrap_socket(client_socket,
server_side=True)
            print("[*] Hoàn tất bắt tay SSL/TLS. Kênh truyền tin đã được mã
hóa.")

            # Nhận dữ liệu từ client
            data = secure_socket.recv(1024)
            if data:
                message = data.decode('utf-8')
                print(f"[Client gửi]: {message}")

                # Gửi phản hồi lại cho client
                response = f"Server đã nhận tin nhắn: '{message}' thành công."
                secure_socket.sendall(response.encode('utf-8'))

        except ssl.SSLError as e:
            print(f"[-] Lỗi bảo mật SSL/TLS trong quá trình bắt tay: {e}")
        except Exception as e:
            print(f"[-] Lỗi xử lý kết nối: {e}")
        finally:
            # Đóng kết nối
            secure_socket.close()
            print(f"[-] Đã đóng kết nối an toàn.")

```

```
except KeyboardInterrupt:
    print("\nServer đang dừng hoạt động...")
finally:
    bind_socket.close()
    print("Server đã dừng.")

if __name__ == "__main__":
    main()
```

Đường dẫn file: `lab03\api_server.py`

```
import sys
import os
from flask import Flask, request, jsonify

# Ensure the lab03 folder is in path for modules import
sys.path.append(os.path.dirname(os.path.abspath(__file__)))

from crypto.rsa_cipher import generate_rsa_keys, rsa_encrypt, rsa_decrypt
from crypto.ecc_signature import generate_ecc_keys, ecc_sign, ecc_verify

app = Flask(__name__)

# Global variables to store keys in memory
rsa_private_key = None
rsa_public_key = None

ecc_private_key = None
ecc_public_key = None

@app.route('/', methods=['GET'])
def index():
    return jsonify({
        "message": "Lab03 RSA ECC API is running"
    })

@app.route('/rsa/generate-key', methods=['POST'])
def rsa_generate_key():
    global rsa_private_key, rsa_public_key
    try:
        rsa_private_key, rsa_public_key = generate_rsa_keys()
        return jsonify({
            "message": "RSA key generated successfully",
            "public_key": rsa_public_key,
            "private_key": rsa_private_key
        })
    except Exception as e:
        return jsonify({"error": f"Không thể tạo khóa RSA: {str(e)}"}), 400
```

```

@app.route('/rsa/encrypt', methods=['POST'])
def rsa_encrypt_endpoint():
    global rsa_public_key
    if not rsa_public_key:
        return jsonify({"error": "Khóa công khai RSA chưa được tạo. Vui lòng gọi /rsa/generate-key trước."}), 400

    data = request.get_json(silent=True)
    if not data or 'plaintext' not in data:
        return jsonify({"error": "Thiếu tham số bắt buộc 'plaintext' trong JSON body."}), 400

    plaintext = data['plaintext']
    try:
        ciphertext = rsa_encrypt(plaintext, rsa_public_key)
        return jsonify({
            "plaintext": plaintext,
            "ciphertext": ciphertext
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 400

@app.route('/rsa/decrypt', methods=['POST'])
def rsa_decrypt_endpoint():
    global rsa_private_key
    if not rsa_private_key:
        return jsonify({"error": "Khóa bí mật RSA chưa được tạo. Vui lòng gọi /rsa/generate-key trước."}), 400

    data = request.get_json(silent=True)
    if not data or 'ciphertext' not in data:
        return jsonify({"error": "Thiếu tham số bắt buộc 'ciphertext' trong JSON body."}), 400

    ciphertext = data['ciphertext']
    try:
        decrypted_text = rsa_decrypt(ciphertext, rsa_private_key)
        return jsonify({
            "decrypted_text": decrypted_text
        })
    except Exception as e:
        return jsonify({"error": f"Giải mã thất bại: {str(e)}"}), 400

@app.route('/ecc/generate-key', methods=['POST'])
def ecc_generate_key_endpoint():
    global ecc_private_key, ecc_public_key
    try:
        ecc_private_key, ecc_public_key = generate_ecc_keys()
        return jsonify({
            "message": "ECC key generated successfully",
            "public_key": ecc_public_key,
            "private_key": ecc_private_key
        })
    
```

```

        except Exception as e:
            return jsonify({"error": f"Không thể tạo khóa ECC: {str(e)}"}), 400

@app.route('/ecc/sign', methods=['POST'])
def ecc_sign_endpoint():
    global ecc_private_key
    if not ecc_private_key:
        return jsonify({"error": "Khóa bí mật ECC chưa được tạo. Vui lòng gọi /ecc/generate-key trước."}), 400

    data = request.get_json(silent=True)
    if not data or 'message' not in data:
        return jsonify({"error": "Thiếu tham số bắt buộc 'message' trong JSON body."}), 400

    message = data['message']
    try:
        signature = ecc_sign(message, ecc_private_key)
        return jsonify({
            "message": message,
            "signature": signature
        })
    except Exception as e:
        return jsonify({"error": str(e)}), 400

@app.route('/ecc/verify', methods=['POST'])
def ecc_verify_endpoint():
    global ecc_public_key
    if not ecc_public_key:
        return jsonify({"error": "Khóa công khai ECC chưa được tạo. Vui lòng gọi /ecc/generate-key trước."}), 400

    data = request.get_json(silent=True)
    if not data:
        return jsonify({"error": "Thiếu dữ liệu JSON body."}), 400

    if 'message' not in data:
        return jsonify({"error": "Thiếu tham số bắt buộc 'message' trong JSON body."}), 400
    if 'signature' not in data:
        return jsonify({"error": "Thiếu tham số bắt buộc 'signature' trong JSON body."}), 400

    message = data['message']
    signature = data['signature']

    try:
        is_valid = ecc_verify(message, signature, ecc_public_key)
        if is_valid:
            return jsonify({
                "valid": True,
                "message": "Signature is valid"
            })
        else:

```

```

        return jsonify({
            "valid": False,
            "message": "Signature is invalid"
        })
    except Exception as e:
        return jsonify({"error": f"Xác minh thất bại: {str(e)}"}), 400

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=5000, debug=True)

```

Đường dẫn file: **lab03\main.py**

```

import sys
import os
from PyQt5.QtWidgets import QApplication, QWidget
from PyQt5 import uic

# Ensure the lab03 folder is in path for modules import
sys.path.append(os.path.dirname(os.path.abspath(__file__)))

from ui.rsa_window import RSAWindow
from ui.ecc_window import ECCWindow

class MainApplication(QWidget):
    def __init__(self):
        super().__init__()

        # Load the UI file dynamically
        ui_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'ui',
                                'main.ui')
        uic.loadUi(ui_path, self)

        self.rsa_window = None
        self.ecc_window = None

        self.init_connections()

    def init_connections(self):
        self.btn_rsa.clicked.connect(self.open_rsa_window)
        self.btn_ecc.clicked.connect(self.open_ecc_window)

    def open_rsa_window(self):
        if not self.rsa_window:
            self.rsa_window = RSAWindow()
            self.rsa_window.show()
            self.rsa_window.raise_()
            self.rsa_window.activateWindow()

    def open_ecc_window(self):

```

```
        if not self.ecc_window:
            self.ecc_window = ECCWindow()
        self.ecc_window.show()
        self.ecc_window.raise_()
        self.ecc_window.activateWindow()

def main():
    app = QApplication(sys.argv)

    # Premium Dark Theme Stylesheet
    dark_stylesheet = """
QWidget {
    background-color: #1e1e24;
    color: #e4e4e6;
    font-family: 'Segoe UI', Helvetica, Arial, sans-serif;
}

QTextEdit, QLineEdit {
    background-color: #2b2b36;
    border: 1px solid #3f3f52;
    border-radius: 6px;
    padding: 8px;
    color: #ffffff;
    selection-background-color: #5865f2;
}

QTextEdit:focus, QLineEdit:focus {
    border: 1px solid #5865f2;
}

QPushButton {
    background-color: #5865f2;
    border: none;
    border-radius: 6px;
    color: white;
    font-weight: bold;
    padding: 10px 18px;
}

QPushButton:hover {
    background-color: #4752c4;
}

QPushButton:pressed {
    background-color: #3c45a5;
}

QLabel {
    font-weight: bold;
    color: #b9bbbe;
}

QMessageBox {
    background-color: #2f3136;
```

```
        color: #ffffff;
    }

    QMessageBox QLabel {
        color: #ffffff;
        font-weight: normal;
    }

    QMessageBox QPushButton {
        background-color: #4f545c;
        min-width: 80px;
    }

    QMessageBox QPushButton:hover {
        background-color: #686d73;
    }
"""

app.setStyleSheet(dark_stylesheet)

window = MainApplication()
window.show()
sys.exit(app.exec_())

if __name__ == "__main__":
    main()
```

Đường dẫn file: `lab03\requirements.txt`

```
PyQt5
pycryptodome
Flask
```

Đường dẫn file: `lab03\crypto\__init__.py`

```
# Crypto package
```

Đường dẫn file: `lab03\crypto\ecc_signature.py`

```

from Crypto.PublicKey import ECC
from Crypto.Signature import DSS
from Crypto.Hash import SHA256
import base64

def generate_ecc_keys():
    """
    Generate ECC key pair using P-256 curve.
    Returns:
        tuple: (private_key_pem, public_key_pem) as strings.
    """
    key = ECC.generate(curve='P-256')
    private_key_pem = key.export_key(format='PEM')
    public_key_pem = key.public_key().export_key(format='PEM')
    return private_key_pem, public_key_pem

def ecc_sign(message: str, private_key_pem: str) -> str:
    """
    Sign a message using the ECC private key and SHA256 + DSS/ECDSA.
    Args:
        message (str): Message to sign.
        private_key_pem (str): ECC private key in PEM format.
    Returns:
        str: Signature in Base64 encoded format.
    """
    try:
        key = ECC.import_key(private_key_pem)
        h = SHA256.new(message.encode('utf-8'))
        signer = DSS.new(key, 'fips-186-3')
        signature = signer.sign(h)
        return base64.b64encode(signature).decode('utf-8')
    except Exception as e:
        raise ValueError(f"Ký thông điệp thất bại: {str(e)}")

def ecc_verify(message: str, signature_b64: str, public_key_pem: str) -> bool:
    """
    Verify a signature using the ECC public key and SHA256 + DSS/ECDSA.
    Args:
        message (str): Original or modified message.
        signature_b64 (str): Base64 encoded signature.
        public_key_pem (str): ECC public key in PEM format.
    Returns:
        bool: True if signature is valid, False otherwise.
    """
    try:
        key = ECC.import_key(public_key_pem)
        h = SHA256.new(message.encode('utf-8'))
        verifier = DSS.new(key, 'fips-186-3')
        signature = base64.b64decode(signature_b64.encode('utf-8'))
        verifier.verify(h, signature)
        return True
    except (ValueError, TypeError, Exception):

```

```
return False
```

Đường dẫn file: `lab03\crypto\rsa_cipher.py`

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import base64

def generate_rsa_keys():
    """
    Generate RSA 2048-bit key pair.
    Returns:
        tuple: (private_key_pem, public_key_pem) as strings.
    """
    key = RSA.generate(2048)
    private_key_pem = key.export_key().decode('utf-8')
    public_key_pem = key.publickey().export_key().decode('utf-8')
    return private_key_pem, public_key_pem

def rsa_encrypt(plaintext: str, public_key_pem: str) -> str:
    """
    Encrypt plaintext using the RSA public key (PKCS1_OAEP).
    Args:
        plaintext (str): The text to encrypt.
        public_key_pem (str): RSA public key in PEM format.
    Returns:
        str: Ciphertext in Base64 encoded format.
    """
    try:
        recipient_key = RSA.import_key(public_key_pem)
        cipher_rsa = PKCS1_OAEP.new(recipient_key)
        ciphertext = cipher_rsa.encrypt(plaintext.encode('utf-8'))
        return base64.b64encode(ciphertext).decode('utf-8')
    except Exception as e:
        raise ValueError(f"Mã hóa thất bại: {str(e)}")

def rsa_decrypt(ciphertext_b64: str, private_key_pem: str) -> str:
    """
    Decrypt Base64 ciphertext using the RSA private key (PKCS1_OAEP).
    Args:
        ciphertext_b64 (str): Base64 encoded ciphertext.
        private_key_pem (str): RSA private key in PEM format.
    Returns:
        str: Decrypted plaintext.
    """
    try:
        private_key = RSA.import_key(private_key_pem)
        cipher_rsa = PKCS1_OAEP.new(private_key)
        ciphertext = base64.b64decode(ciphertext_b64.encode('utf-8'))
```

```
        decrypted = cipher_rsa.decrypt(ciphertext)
        return decrypted.decode('utf-8')
    except Exception as e:
        raise ValueError(f"Giải mã thất bại. Vui lòng kiểm tra lại khóa hoặc bản mã: {str(e)}")
```

Đường dẫn file: lab03\ui__init__.py

```
# UI package
```

Đường dẫn file: lab03\ui\ecc.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
    <class>ECCWidget</class>
    <widget class="QWidget" name="ECCWidget">
        <property name="geometry">
            <rect>
                <x>0</x>
                <y>0</y>
                <width>800</width>
                <height>600</height>
            </rect>
        </property>
        <property name="windowTitle">
            <string>ECC Digital Signature</string>
        </property>
        <layout class="QVBoxLayout" name="mainVerticalLayout">
            <property name="spacing">
                <number>15</number>
            </property>
            <property name="leftMargin">
                <number>20</number>
            </property>
            <property name="topMargin">
                <number>20</number>
            </property>
            <property name="rightMargin">
                <number>20</number>
            </property>
            <property name="bottomMargin">
                <number>20</number>
            </property>
            <item>
                <widget class="QLabel" name="title_label">
```

```

        <property name="text">
            <string>ECC P-256 Digital Signature (ECDSA)</string>
        </property>
        <property name="styleSheet">
            <string>font-size: 18px; font-weight: bold; color: #5865f2; margin-bottom:
10px;</string>
        </property>
    </widget>
</item>
<item>
    <layout class="QGridLayout" name="keysGridLayout">
        <property name="spacing">
            <number>10</number>
        </property>
        <item row="0" column="0">
            <widget class="QLabel" name="pubLabel">
                <property name="text">
                    <string>Public Key (PEM):</string>
                </property>
            </widget>
        </item>
        <item row="1" column="0">
            <widget class="QTextEdit" name="pub_key_txt">
                <property name="readOnly">
                    <bool>true</bool>
                </property>
                <property name="placeholderText">
                    <string>Khóa công khai (Public Key) sẽ hiển thị ở đây...</string>
                </property>
                <property name="maximumHeight">
                    <number>150</number>
                </property>
            </widget>
        </item>
        <item row="0" column="1">
            <widget class="QLabel" name="privLabel">
                <property name="text">
                    <string>Private Key (PEM):</string>
                </property>
            </widget>
        </item>
        <item row="1" column="1">
            <widget class="QTextEdit" name="priv_key_txt">
                <property name="readOnly">
                    <bool>true</bool>
                </property>
                <property name="placeholderText">
                    <string>Khóa bí mật (Private Key) sẽ hiển thị ở đây...</string>
                </property>
                <property name="maximumHeight">
                    <number>150</number>
                </property>
            </widget>
        </item>
    </layout>
</item>

```

```

</layout>
</item>
<item>
  <widget class="QPushButton" name="btn_gen_key">
    <property name="text">
      <string>Generate ECC Key</string>
    </property>
    <property name="cursor">
      <cursorShape>PointingHandCursor</cursorShape>
    </property>
    <property name="styleSheet">
      <string>padding: 10px; font-size: 14px;</string>
    </property>
  </widget>
</item>
<item>
  <widget class="QLabel" name="msgLabel">
    <property name="text">
      <string>Message (Thông điệp):</string>
    </property>
  </widget>
</item>
<item>
  <widget class="QTextEdit" name="message_txt">
    <property name="placeholderText">
      <string>Nhập nội dung cần ký...</string>
    </property>
    <property name="maximumHeight">
      <number>100</number>
    </property>
  </widget>
</item>
<item>
  <layout class="QHBoxLayout" name="actionsHorizontalLayout">
    <property name="spacing">
      <number>20</number>
    </property>
    <item>
      <widget class="QPushButton" name="btn_sign">
        <property name="text">
          <string>Sign Message (Ký chữ ký)</string>
        </property>
        <property name="cursor">
          <cursorShape>PointingHandCursor</cursorShape>
        </property>
      </widget>
    </item>
    <item>
      <widget class="QPushButton" name="btn_verify">
        <property name="text">
          <string>Verify Signature (Xác minh)</string>
        </property>
        <property name="cursor">
          <cursorShape>PointingHandCursor</cursorShape>
        </property>
      </widget>
    </item>
  </layout>
</item>

```

```

        </property>
    </widget>
</item>
</layout>
</item>
<item>
    <widget class="QLabel" name="sigLabel">
        <property name="text">
            <string>Signature (Base64) (Chữ ký):</string>
        </property>
    </widget>
</item>
<item>
    <widget class="QTextEdit" name="signature_txt">
        <property name="placeholderText">
            <string>Chữ ký số dạng Base64...</string>
        </property>
        <property name="maximumHeight">
            <number>80</number>
        </property>
    </widget>
</item>
<item>
    <layout class="QHBoxLayout" name="statusHorizontalLayout">
        <item>
            <widget class="QLabel" name="statusLabel">
                <property name="text">
                    <string>Kết quả xác minh (Verification Result):</string>
                </property>
            </widget>
        </item>
        <item>
            <widget class="QLineEdit" name="status_val">
                <property name="readOnly">
                    <bool>true</bool>
                </property>
                <property name="placeholderText">
                    <string>Chưa thực hiện xác minh</string>
                </property>
                <property name="styleSheet">
                    <string>font-weight: bold; font-size: 14px; text-align: center; padding:
5px;</string>
                </property>
            </widget>
        </item>
    </layout>
</item>
<item>
    <widget class="QPushButton" name="btn_back">
        <property name="text">
            <string>Quay lại Menu</string>
        </property>
        <property name="styleSheet">
            <string>background-color: #4f545c; font-weight: normal; margin-top: 10px;

```

```

</string>
    </property>
</widget>
</item>
</layout>
</widget>
<resources/>
<connections/>
</ui>

```

Đường dẫn file: `lab03\ui\ecc_window.py`

```

import os
from PyQt5.QtWidgets import QWidget, QMessageBox
from PyQt5 import uic
from crypto.ecc_signature import generate_ecc_keys, ecc_sign, ecc_verify

class ECCWindow(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)

        # Load the UI file dynamically
        ui_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                                'ecc.ui')
        uic.loadUi(ui_path, self)

        # Local keys storage
        self.private_key = None
        self.public_key = None

        self.init_connections()

    def init_connections(self):
        # Connect buttons defined in ecc.ui to handler methods
        self.btn_gen_key.clicked.connect(self.handle_generate_key)
        self.btn_sign.clicked.connect(self.handle_sign)
        self.btn_verify.clicked.connect(self.handle_verify)
        self.btn_back.clicked.connect(self.close)

    def handle_generate_key(self):
        try:
            self.private_key, self.public_key = generate_ecc_keys()
            self.pub_key_txt.setPlainText(self.public_key)
            self.priv_key_txt.setPlainText(self.private_key)
            QMessageBox.information(self, "Thành công", "Sinh cặp khóa ECC P-256 thành công!")
        except Exception as e:
            QMessageBox.critical(self, "Lỗi", f"Không thể sinh cặp khóa: {str(e)}")

```

```

def handle_sign(self):
    if not self.private_key:
        QMessageBox.warning(self, "Lỗi", "Vui lòng sinh khóa ECC trước khi thực hiện ký!")
        return

    message = self.message_txt.toPlainText()
    if not message:
        QMessageBox.warning(self, "Thông báo", "Vui lòng nhập Message cần ký!")
        return

    try:
        signature = ecc_sign(message, self.private_key)
        self.signature_txt.setPlainText(signature)
        self.status_val.clear()
        self.status_val.setPlaceholderText("Chưa thực hiện xác minh")
        self.status_val.setStyleSheet("font-weight: bold; font-size: 14px; padding: 5px; color: #b9bbbe; background-color: #2b2b36;")
    except Exception as e:
        QMessageBox.critical(self, "Lỗi ký", str(e))

def handle_verify(self):
    if not self.public_key:
        QMessageBox.warning(self, "Lỗi", "Vui lòng sinh khóa ECC trước khi xác minh chữ ký!")
        return

    message = self.message_txt.toPlainText()
    signature = self.signature_txt.toPlainText().strip()

    if not signature:
        QMessageBox.warning(self, "Thông báo", "Vui lòng có chữ ký (Signature) để xác minh!")
        return

    is_valid = ecc_verify(message, signature, self.public_key)

    if is_valid:
        self.status_val.setText("Chữ ký HỢP LỆ (Signature is Valid)")
        self.status_val.setStyleSheet("font-weight: bold; font-size: 14px; padding: 5px; color: #43b581; background-color: #2b2b36; border: 1px solid #43b581;")
        QMessageBox.information(self, "Kết quả xác minh", "Chữ ký hợp lệ!")
    else:
        self.status_val.setText("Chữ ký KHÔNG HỢP LỆ (Signature is Invalid)")
        self.status_val.setStyleSheet("font-weight: bold; font-size: 14px; padding: 5px; color: #f04747; background-color: #2b2b36; border: 1px solid #f04747;")
        QMessageBox.warning(self, "Kết quả xác minh", "Chữ ký KHÔNG hợp lệ! Bản tin có thể đã bị sửa đổi hoặc chữ ký không đúng.")

```

Đường dẫn file: lab03\ui\main.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
  <class>MainWindow</class>
  <widget class="QWidget" name="MainWindow">
    <property name="geometry">
      <rect>
        <x>0</x>
        <y>0</y>
        <width>500</width>
        <height>380</height>
      </rect>
    </property>
    <property name="windowTitle">
      <string>Lab 03 - RSA & ECC Security Application</string>
    </property>
    <layout class="QVBoxLayout" name="verticalLayout">
      <property name="spacing">
        <number>20</number>
      </property>
      <property name="leftMargin">
        <number>30</number>
      </property>
      <property name="topMargin">
        <number>30</number>
      </property>
      <property name="rightMargin">
        <number>30</number>
      </property>
      <property name="bottomMargin">
        <number>30</number>
      </property>
      <item>
        <widget class="QLabel" name="title_label">
          <property name="text">
            <string>Lab 03 - RSA & ECC Security Application</string>
          </property>
          <property name="alignment">
            <set>Qt::AlignCenter</set>
          </property>
          <property name="styleSheet">
            <string>font-size: 20px; font-weight: bold; color: #5865f2; margin-bottom:
5px;</string>
          </property>
        </widget>
      </item>
      <item>
        <widget class="QLabel" name="author_label">
          <property name="text">
```

```

        <string>Sinh viên: Lê Việt Đức - MSSV: 0533 - Lớp: 23DTHA4</string>
    </property>
    <property name="alignment">
        <set>Qt::AlignCenter</set>
    </property>
    <property name="styleSheet">
        <string>font-size: 13px; color: #b9bbbe;</string>
    </property>
</widget>
</item>
<item>
    <widget class="QLabel" name="course_label">
        <property name="text">
            <string>Môn: Thực hành Bảo mật thông tin nâng cao</string>
        </property>
        <property name="alignment">
            <set>Qt::AlignCenter</set>
        </property>
        <property name="styleSheet">
            <string>font-size: 13px; color: #b9bbbe; font-style: italic;</string>
        </property>
    </widget>
</item>
<item>
    <widget class="QPushButton" name="btn_rsa">
        <property name="text">
            <string>1. RSA Encryption / Decryption</string>
        </property>
        <property name="cursor">
            <cursorShape>PointingHandCursor</cursorShape>
        </property>
        <property name="styleSheet">
            <string>padding: 15px; font-size: 14px; text-align: left; padding-left:
25px;</string>
        </property>
    </widget>
</item>
<item>
    <widget class="QPushButton" name="btn_ecc">
        <property name="text">
            <string>2. ECC Digital Signature</string>
        </property>
        <property name="cursor">
            <cursorShape>PointingHandCursor</cursorShape>
        </property>
        <property name="styleSheet">
            <string>padding: 15px; font-size: 14px; text-align: left; padding-left:
25px;</string>
        </property>
    </widget>
</item>
<item>
    <widget class="QLabel" name="footer_label">
        <property name="text">

```

```

        <string>Chọn một chức năng để bắt đầu</string>
    </property>
    <property name="alignment">
        <set>Qt::AlignCenter</set>
    </property>
    <property name="styleSheet">
        <string>color: #72767d; font-size: 11px;</string>
    </property>
</widget>
</item>
</layout>
</widget>
<resources/>
<connections/>
</ui>

```

Đường dẫn file: lab03\ui\rsa.ui

```

<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
    <class>RSAWidget</class>
    <widget class="QWidget" name="RSAWidget">
        <property name="geometry">
            <rect>
                <x>0</x>
                <y>0</y>
                <width>800</width>
                <height>600</height>
            </rect>
        </property>
        <property name="windowTitle">
            <string>RSA Encryption / Decryption</string>
        </property>
        <layout class="QVBoxLayout" name="mainVerticalLayout">
            <property name="spacing">
                <number>15</number>
            </property>
            <property name="leftMargin">
                <number>20</number>
            </property>
            <property name="topMargin">
                <number>20</number>
            </property>
            <property name="rightMargin">
                <number>20</number>
            </property>
            <property name="bottomMargin">
                <number>20</number>
            </property>

```

```

<item>
  <widget class="QLabel" name="title_label">
    <property name="text">
      <string>RSA 2048-bit Encryption & Decryption</string>
    </property>
    <property name="styleSheet">
      <string>font-size: 18px; font-weight: bold; color: #5865f2; margin-bottom:
10px;</string>
    </property>
  </widget>
</item>
<item>
  <layout class="QGridLayout" name="keysGridLayout">
    <property name="spacing">
      <number>10</number>
    </property>
    <item row="0" column="0">
      <widget class="QLabel" name="pubLabel">
        <property name="text">
          <string>Public Key (PEM):</string>
        </property>
      </widget>
    </item>
    <item row="1" column="0">
      <widget class="QTextEdit" name="pub_key_txt">
        <property name="readOnly">
          <bool>true</bool>
        </property>
        <property name="placeholderText">
          <string>Khóa công khai (Public Key) sẽ hiển thị ở đây...</string>
        </property>
        <property name="maximumHeight">
          <number>150</number>
        </property>
      </widget>
    </item>
    <item row="0" column="1">
      <widget class="QLabel" name="privLabel">
        <property name="text">
          <string>Private Key (PEM):</string>
        </property>
      </widget>
    </item>
    <item row="1" column="1">
      <widget class="QTextEdit" name="priv_key_txt">
        <property name="readOnly">
          <bool>true</bool>
        </property>
        <property name="placeholderText">
          <string>Khóa bí mật (Private Key) sẽ hiển thị ở đây...</string>
        </property>
        <property name="maximumHeight">
          <number>150</number>
        </property>
      </widget>
    </item>
  </layout>
</item>

```

```

        </widget>
    </item>
</layout>
</item>
<item>
    <widget class="QPushButton" name="btn_gen_key">
        <property name="text">
            <string>Generate RSA Key</string>
        </property>
        <property name="cursor">
            <cursorShape>PointingHandCursor</cursorShape>
        </property>
        <property name="styleSheet">
            <string>padding: 10px; font-size: 14px;</string>
        </property>
    </widget>
</item>
<item>
    <layout class="QHBoxLayout" name="contentHorizontalLayout">
        <property name="spacing">
            <number>20</number>
        </property>
        <item>
            <layout class="QVBoxLayout" name="leftVerticalLayout">
                <item>
                    <widget class="QLabel" name="plainLabel">
                        <property name="text">
                            <string>Plaintext (Bản rõ):</string>
                        </property>
                    </widget>
                </item>
                <item>
                    <widget class="QTextEdit" name="plaintext_txt">
                        <property name="placeholderText">
                            <string>Nhập nội dung cần mã hóa...</string>
                        </property>
                    </widget>
                </item>
                <item>
                    <widget class="QPushButton" name="btn_encrypt">
                        <property name="text">
                            <string>Encrypt (Mã hóa)</string>
                        </property>
                        <property name="cursor">
                            <cursorShape>PointingHandCursor</cursorShape>
                        </property>
                    </widget>
                </item>
            </layout>
        </item>
        <item>
            <layout class="QVBoxLayout" name="rightVerticalLayout">
                <item>
                    <widget class="QLabel" name="cipherLabel">

```

```

        <property name="text">
            <string>Ciphertext (Base64) (Bản mã):</string>
        </property>
    </widget>
</item>
<item>
    <widget class="QTextEdit" name="ciphertext_txt">
        <property name="placeholderText">
            <string>Bản mã dạng Base64...</string>
        </property>
    </widget>
</item>
<item>
    <widget class="QPushButton" name="btn_decrypt">
        <property name="text">
            <string>Decrypt (Giải mã)</string>
        </property>
        <property name="cursor">
            <cursorShape>PointingHandCursor</cursorShape>
        </property>
    </widget>
</item>
<item>
    <widget class="QLabel" name="decryptedLabel">
        <property name="text">
            <string>Decrypted Text (Bản giải mã):</string>
        </property>
    </widget>
</item>
<item>
    <widget class="QTextEdit" name="decrypted_txt">
        <property name="readOnly">
            <bool>true</bool>
        </property>
        <property name="placeholderText">
            <string>Nội dung sau khi giải mã...</string>
        </property>
        <property name="maximumHeight">
            <number>100</number>
        </property>
    </widget>
</item>
</layout>
</item>
</layout>
</item>
<item>
    <widget class="QPushButton" name="btn_back">
        <property name="text">
            <string>Quay lại Menu</string>
        </property>
        <property name="styleSheet">
            <string>background-color: #4f545c; font-weight: normal; margin-top: 10px;
</string>

```

```

        </property>
    </widget>
</item>
</layout>
</widget>
<resources/>
<connections/>
</ui>

```

Đường dẫn file: `lab03\ui\rsa_window.py`

```

import os
from PyQt5.QtWidgets import QWidget, QMessageBox
from PyQt5 import uic
from crypto.rsa_cipher import generate_rsa_keys, rsa_encrypt, rsa_decrypt

class RSAWindow(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)

        # Load the UI file dynamically
        ui_path = os.path.join(os.path.dirname(os.path.abspath(__file__)),
'rsa.ui')
        uic.loadUi(ui_path, self)

        # Local keys storage
        self.private_key = None
        self.public_key = None

        self.init_connections()

    def init_connections(self):
        # Connect buttons defined in rsa.ui to handler methods
        self.btn_gen_key.clicked.connect(self.handle_generate_key)
        self.btn_encrypt.clicked.connect(self.handle_encrypt)
        self.btn_decrypt.clicked.connect(self.handle_decrypt)
        self.btn_back.clicked.connect(self.close)

    def handle_generate_key(self):
        try:
            self.private_key, self.public_key = generate_rsa_keys()
            self.pub_key_txt.setPlainText(self.public_key)
            self.priv_key_txt.setPlainText(self.private_key)
            QMessageBox.information(self, "Thành công", "Sinh cặp khóa RSA 2048-bit thành công!")
        except Exception as e:
            QMessageBox.critical(self, "Lỗi", f"Không thể sinh cặp khóa: {str(e)}")

```

```
def handle_encrypt(self):
    if not self.public_key:
        QMessageBox.warning(self, "Lỗi", "Vui lòng sinh khóa RSA trước khi
thực hiện mã hóa!")
        return

    plaintext = self.plaintext_txt.toPlainText()
    if not plaintext:
        QMessageBox.warning(self, "Thông báo", "Vui lòng nhập Plaintext cần mã
hóa!")
        return

    try:
        ciphertext = rsa_encrypt(plaintext, self.public_key)
        self.ciphertext_txt.setPlainText(ciphertext)
    except Exception as e:
        QMessageBox.critical(self, "Lỗi mã hóa", str(e))

def handle_decrypt(self):
    if not self.private_key:
        QMessageBox.warning(self, "Lỗi", "Vui lòng sinh khóa RSA trước khi
thực hiện giải mã!")
        return

    ciphertext = self.ciphertext_txt.toPlainText().strip()
    if not ciphertext:
        QMessageBox.warning(self, "Thông báo", "Vui lòng nhập Ciphertext cần
giải mã!")
        return

    try:
        decrypted = rsa_decrypt(ciphertext, self.private_key)
        self.decrypted_txt.setPlainText(decrypted)
    except Exception as e:
        self.decrypted_txt.setPlainText("Lỗi giải mã")
        QMessageBox.warning(self, "Lỗi giải mã", "Giải mã không thành công!
Bản mã đã bị thay đổi hoặc khóa không khớp.")
```